

Class 4: Feature Engineering and Data Representation

Data-driven Systems Engineering

Agenda

- Why feature engineering often matters more than model complexity.
- Cleaning, curation, selection, extraction, and construction.
- Encoding and transforming categorical, numerical, temporal, and streaming data.
- Reproducible feature pipelines and leakage prevention.
- Repository examples from fraud detection and pasteurization monitoring.

Learning goals

- Distinguish feature engineering subtasks clearly.
- Choose suitable encodings and transformations for different variable types.
- Recognize leakage and feature availability problems.
- Explain why feature logic must be reproducible across training and serving.

Course Map and Running Cases



Today in the course

Focus: Feature representation and reproducible preprocessing.

Bridge: This class converts exploratory findings into stable model inputs and prepares the handoff from data analysis to modeling.

Fraud case

In the fraud case, feature work means encoding transaction behavior, preventing leakage, and keeping preprocessing consistent across training and inference.

Pasteurization case

In the pasteurization case, feature work includes rates of change, state-aware variables, and derived indicators such as dTdt for process monitoring.

Course thread: After this point, the course can talk about model quality because the representation problem has been made explicit.

Why features matter so much

- Models only learn from the representation we give them.
- Better features can simplify the modeling problem dramatically.
- Bad features can hide leakage, inject bias, or fail at inference time.
- Feature quality usually determines whether deployment is robust or fragile.

Five related but different tasks

Cleaning	Fix missingness, duplicates, invalid values, and basic errors.
Curation	Organize, document, and validate datasets for reuse.
Selection	Keep the most informative existing variables.
Extraction	Transform variables into a new lower-dimensional representation.
Construction	Build new features from domain logic, time, aggregation, or combinations.

Richer examples than textbook toy cases

- Fraud detection: rolling transaction count, deviation from recent spending, merchant risk history.
- Maintenance or process monitoring: moving average, slope, rate of change, state duration.
- E-commerce: time since last purchase, category diversity, and promotion exposure.
- Healthcare scheduling: no-show history, appointment lead time, and clinic congestion.

Encoding and transformation choices

One-hot encoding	Good for low-cardinality categories.
Ordinal encoding	Use only when order is real and meaningful.
Target encoding	Useful for high-cardinality categories, but must control leakage.
Scaling	Important for PCA, distance-based methods, and gradient-based models.
Cyclic encoding	Useful for hour-of-day, day-of-week, and seasonal variables.

Notebook example: missing values and typed filling

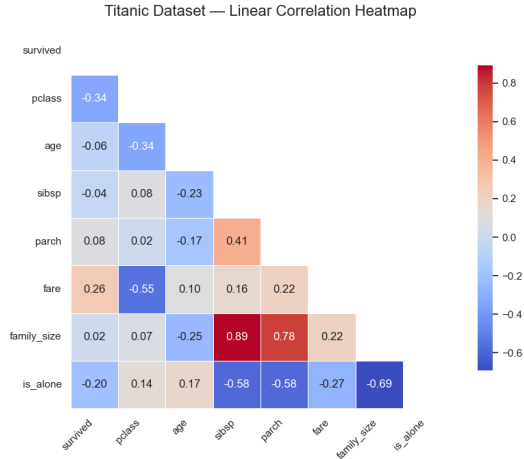
```
titanic = titanic.dropna(subset=["survived"])

for col in ["deck", "embark_town"]:
    titanic[col] = titanic[col].astype("category")
    titanic[col] = titanic[col].cat.add_categories("Unknown")
    titanic[col] = titanic[col].fillna("Unknown")

titanic["age"] = titanic["age"].fillna(titanic["age"].median())
titanic["embarked"] = titanic["embarked"].fillna("S")
```

- This follows 'notebooks/Class4_DataEngineering.ipynb'.
- The main lesson is that missing-value policy depends on column meaning, not on a single global rule.

Notebook plot: correlation before feature decisions



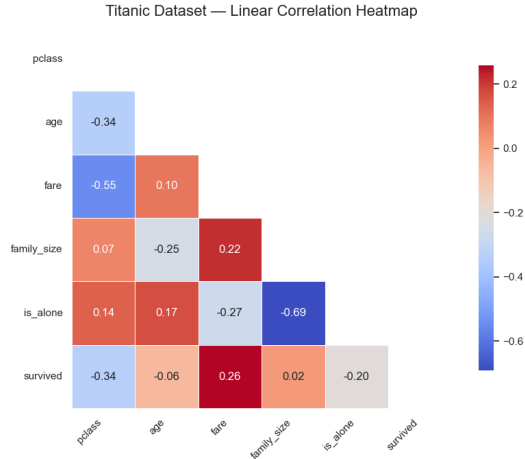
From 'Class4_DataEngineering.ipynb': a first pass at linear relationships in the Titanic variables before selection and construction.

Notebook example: constructing useful features

```
titanic["family_size"] = titanic["sibsp"] + titanic["parch"] + 1
titanic["is_alone"] = (titanic["family_size"] == 1).astype(int)
titanic["title"] = titanic["who"].map(
    {"man": "Mr", "woman": "Mrs", "child": "Master"}
)
```

- These are better examples than generic polynomial features because they reflect simple domain logic.
- Students can discuss which features are causal hints, proxies, or redundant transformations.

Notebook plot: engineered features change the picture



The second heatmap from the notebook makes the new variables visible in relation to the target and the original numeric fields.

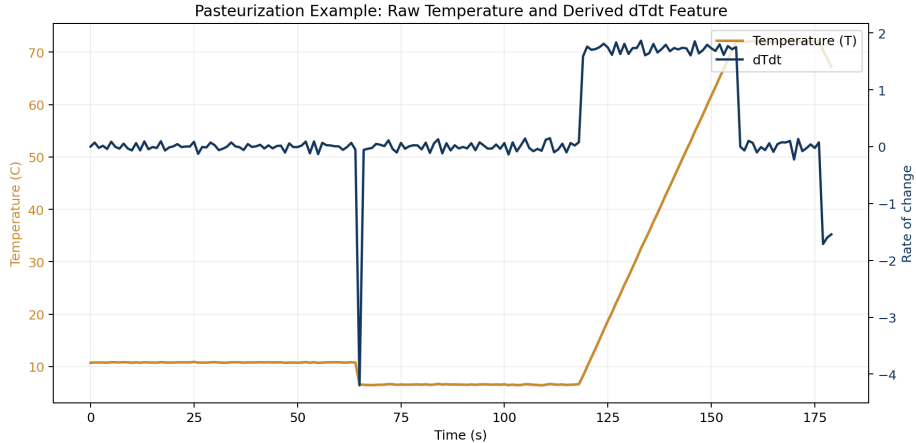
Feature engineering for streams and sensors

- Instantaneous values are often less informative than trends and rates.
- Window-based features such as rolling means can reveal process stability.
- State-aware features can distinguish normal variation from anomalies.
- Streaming systems must compute features online, not only offline.

Repository example

'pasteurization/synth_sensors.py' already includes the derived feature 'dTdt', which is a simple but meaningful process feature.

Stream example: from raw signal to derived feature



In 'pasteurization/synth_sensors.py', 'dTdt' is engineered from temperature to expose heating and cooling behavior more clearly than raw values alone.

Feature pipelines in this repository

Repo activity

- 'notebooks/Class4_DataEngineering.ipynb' introduces tabular transformation logic.
- 'streamlit/modeling.py' and 'streamlit/pages/App_Credit_Card.py' show why the scaler must be saved with the model.
- 'documents/FraudDetectionAsaService.pdf' describes a workflow where transformed variables ultimately feed an API.

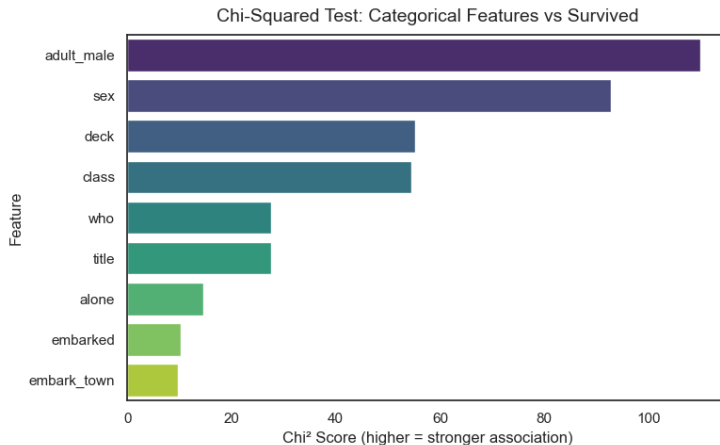
Leakage and availability checks

- Was the feature available at prediction time?
- Does it encode future information, post-decision outcomes, or manual review results?
- Will the feature be computed the same way in production?
- Does the feature depend on unstable joins or missing upstream systems?

Why pipelines matter

- Pipelines make preprocessing explicit and repeatable.
- They reduce mismatch between training and inference.
- They simplify testing, persistence, and model comparison.
- They make handoffs between notebooks and services more reliable.

Notebook plot: chi-squared scores for categorical features



This notebook figure helps explain selection: not every available categorical feature contributes equally to predicting the outcome.

Notebook example: preprocessing as a pipeline

```
numeric_features = ["age", "fare", "family_size"]
categorical_features = ["pclass", "sex", "embarked", "title"]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([("scaler", StandardScaler())]), numeric_features),
        ("cat", Pipeline([("encoder", OneHotEncoder(handle_unknown="ignore"))]),
         categorical_features),
    ]
)
```

- This makes feature logic serializable, testable, and reusable in later modeling and serving stages.
- It also reduces the chance of training-serving skew.

Papers and materials

scikit-learn pipelines and composite estimators; ColumnTransformer; pandas documentation; TFX Transform.

From This Class to the Next

What stays with us

- Features are engineered representations of domain assumptions, not raw columns passed through blindly.
- Pipelines make preprocessing reproducible and reduce training-serving skew.
- Better feature design can outperform more complex model choice.

Project use

Project teams should now be able to justify which variables to create, encode, scale, or discard and how that logic will stay consistent later.

Next connection

Class 5 builds on these representations to compare models, metrics, baselines, and adaptation strategies.

Course thread: Once features represent the problem well, the modeling step becomes a principled comparison instead of guesswork.

Papers and materials

Zheng and Casari, Feature Engineering for Machine Learning; Domingos (2012), A Few Useful Things to Know About Machine Learning; Pedregosa et al. (2011), scikit-learn; Reproducible preprocessing with TFX Transform.